Method Resolution Order (MRO)



KEY IDEAS

- **MRO** = the order Python searches classes for an attribute or method. Python 3 computes it with **C3 linearization**.
- Rules: a class comes **before its parents**; bases keep their **left-to-right** order; each class appears **once**.
- Inspect with `Cls.__mro__`, `Cls.mro()` or `help(Cls)`.
- `super()` means "the **next class in the MRO**", not simply "the parent".

```
      A
     / \
    B   C        D.__mro__ is
     \ /         D, B, C, A, object
      D
```

EXAMPLE

```python
class A:
    def who(self): return "A"

class B(A):
    def who(self): return "B>" + super().who()

class C(A):
    def who(self): return "C>" + super().who()

class D(B, C):
    def who(self): return "D>" + super().who()
```

```python
print([k.__name__ for k in D.__mro__])
# → ['D', 'B', 'C', 'A', 'object']
print(D().who())                # → D>B>C>A

# class X(A, D): ...   (A listed before its subclass D)
# → TypeError: Cannot create a consistent method
#   resolution order (MRO) for bases A, D
```